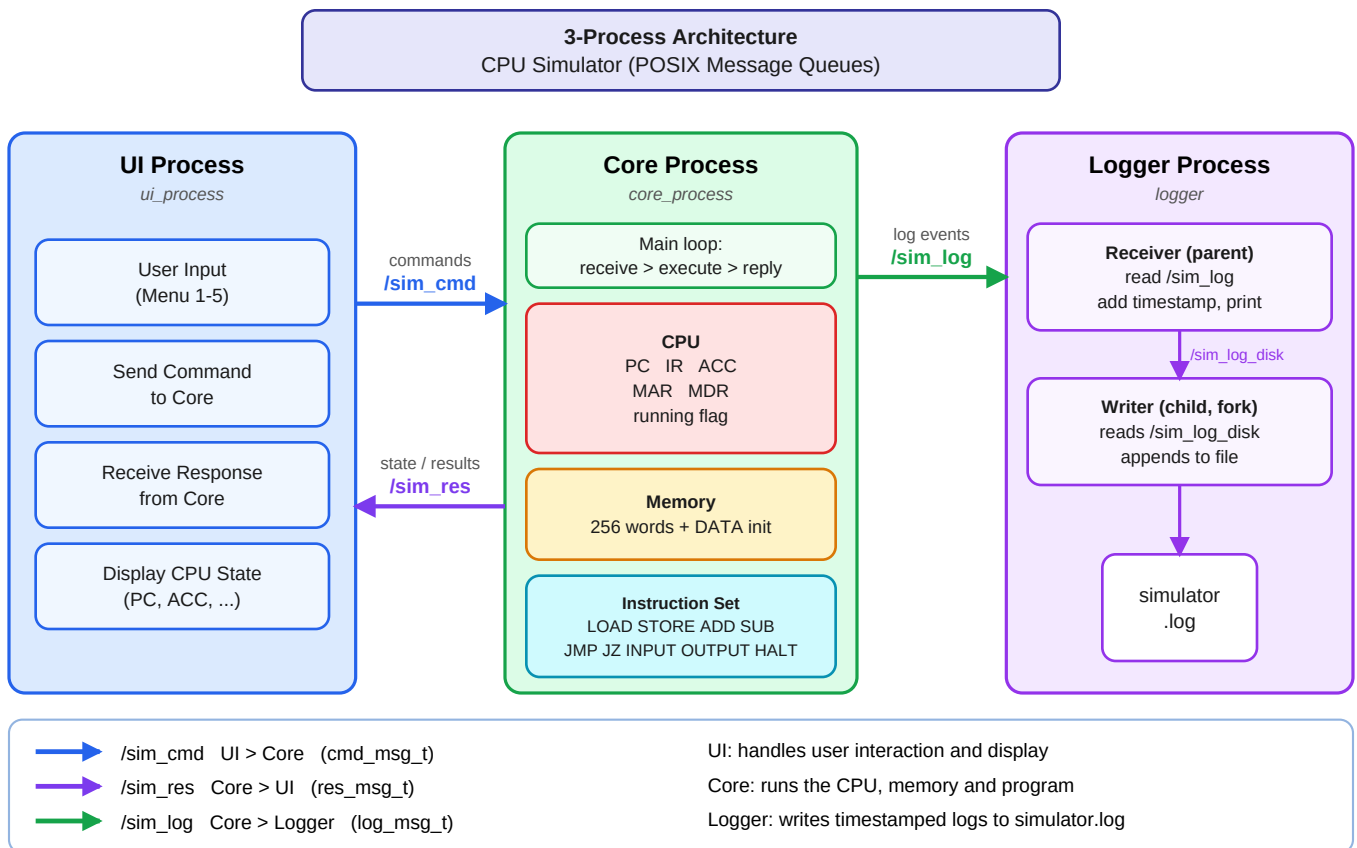


DESIGN 3 PROCESS ARCHITECTURE



Each command from the UI produces exactly one response from the Core, and the Core also sends log events to the Logger. The Logger itself uses a second, internal queue (/sim_log_disk) to hand lines to a child process that writes the file.

How each queue is used

Queue	Direction	Message	Contents
/sim_cmd	UI to Core	cmd_msg_t	Command type (LOAD, RUN, STEP, RESET, QUIT) and a filename for LOAD
/sim_res	Core to UI	res_msg_t	Result type (STATE, HALTED, ERROR), PC, IR, ACC/MAR/MDR, running flag, text
/sim_log	Core to Logger	log_msg_t	Level (INFO, ERROR, QUIT), timestamp, text such as "PC=0 executing LOAD 10"
/sim_log_disk	Logger receiver to Logger writer	disk_msg_t	Formatted log line and a quit marker (internal to the Logger)

Why not the other IPC methods?

Method	Why we did not choose it
Pipes	They carry a raw byte stream, so we would have to mark where each message ends and parse it ourselves. A plain pipe is one-way, so we would need several of them, and processes must be related (anonymous pipes) or we need named FIFOs.
Shared memory	Very fast, but several processes touching the same data need locks or semaphores. That adds complexity and the risk of race conditions, which is unnecessary for small command and log messages.
Sockets	Designed for communication over a network. All three processes run on one machine, so sockets would add address and connection handling for no benefit.
POSIX message queues	Message boundaries are preserved, messages are delivered in order, the kernel handles blocking and waiting, queues have names so processes can start in any order, and timed send/receive lets a process report an error instead of hanging if another process is not running.

What happens when the program runs

- 1. Start-up.** Logger, Core and UI each open the three queues. Whichever starts first creates them, so the start order does not matter.
- 2. Load.** The UI sends CMD_LOAD with a filename on /sim_cmd. The Core checks the program file, stores the instructions and replies on /sim_res.
- 3. Run or step.** The Core repeats fetch, decode and execute: it reads the instruction at PC, uses MAR and MDR for memory access, updates ACC and PC, and sends a log line to the Logger for each instruction. It stops at HALT and sends one final reply to the UI.
- 4. Logging.** The Logger's receiver adds the timestamp, prints the line and passes it to its writer child, which appends it to simulator.log.
- 5. Quit.** The UI sends CMD_QUIT. The Core replies, tells the Logger to flush and exit, removes the queues and exits.